

Java OOP

Java Full Stack Program

Outline

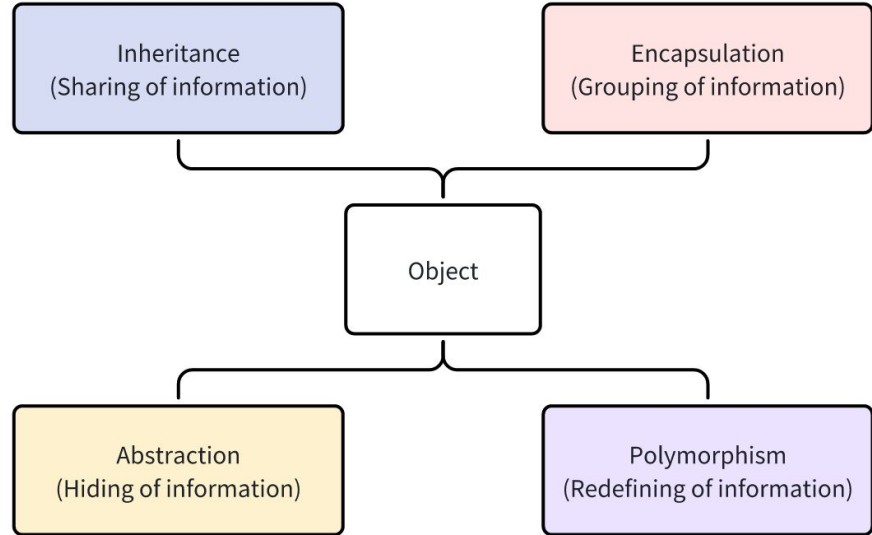
- Object in Java
- Object-Oriented Programming (OOP)
 - Inheritance
 - `java.lang.Object`
 - Different kinds of inheritance
 - Polymorphism
 - Overriding, Overloading
 - Abstraction
 - Abstract class
 - Interface
 - Encapsulation
 - Abstraction vs. Encapsulation
- Immutable class

Object (recap)

- An object is a combination of **data** and **operations** working on the available data
 - Example: a car
- In Java:
 - Data of an object is stored in **fields**
 - **Methods** display the object's behavior
- Class is the blueprint
- Instance: describes the relationship between class and object.
 - Eg: `String s = "Hello world!"` $\Rightarrow$ `s` is an instance of `String`

Object-Oriented Programming (OOP)

- What is OOP
 - A programming paradigm
 - Treating everything as an object
- 4 Pillars
 - Inheritance
 - Polymorphism
 - Abstraction
 - Encapsulation



To Memorize The Four Pillars

- **A PIE**

- Abstraction
- Polymorphism
- Inheritance
- Encapsulation



OOP Pillar: Inheritance

- Is the ability to derive something specific from something generic
 - Animal -> Dog
- Enhances reusability
- A class can inherit the features of another class and add its own modifications
 - Dog: **Subclass** or **Child class**
 - Animal: **Superclass** or **Parent class**
- A subclass inherits **ALL** the *public* and *protected* properties and behaviors of the superclass
 - If subclass is in the same package as the superclass, it also inherits all package-private properties and behaviors
 - A subclass does NOT inherit the *private* members of its superclass

Inheritance in Java

- Syntax
 - `class MySubClass extends MySuperClass`
 - **extends** keyword declares that **MySubClass** inherits the parent class **MySuperClass**
- IS-A relationship
 - Inheritance in Java implies that there is an **IS-A** relationship between subclass and superclass
 - A dog **is** an Animal

Object Class in Java

- The **Object class** is the parent class of all the classes in Java by default.
- It sits at the top of the class hierarchy
 - `java.lang.Object`
 - Object class has no superclass
 - Contains some basic methods describing the common behavior of all Java objects:
 - `equals()`
 - `hashCode()`
 - `toString()`

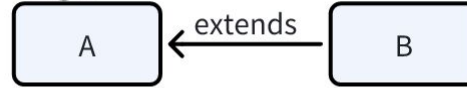
Methods of Object Class

Modifier and Type	Method	Description
protected Object	clone()	Creates and returns a copy of this object.
boolean	equals(Object obj)	Indicates whether some other object is "equal to" this one.
protected void	finalize()	Called by the garbage collector on an object when garbage collection determines that there are no more references to the object.
Class<?>	getClass()	Returns the runtime class of this Object .
int	hashCode()	Returns a hash code value for the object.
void	notify()	Wakes up a single thread that is waiting on this object's monitor.
void	notifyAll()	Wakes up all threads that are waiting on this object's monitor.
String	toString()	Returns a string representation of the object.
void	wait()	Causes the current thread to wait until another thread invokes the notify() method or the notifyAll() method for this object.
void	wait(long timeout)	Causes the current thread to wait until either another thread invokes the notify() method or the notifyAll() method for this object, or a specified amount of time has elapsed.
void	wait(long timeout, int nanos)	Causes the current thread to wait until another thread invokes the notify() method or the notifyAll() method for this object, or some other thread interrupts the current thread, or a certain amount of real time has elapsed.

Types of Inheritance

- **Single**
 - Inherit from only one class
- **Multiple***
 - Not supported by Java
 - Inherit from more than one unrelated class
- **Multi-Level**
 - No limit to the chain of inheritance
 - Makes code excessively complex if too deep
- **Hierarchical**
 - More than one class can inherit from a single class
- **Hybrid**
 - Any combination of the above

Single



Multiple (Not Supported By Java)



Multi-Level



Hierarchical

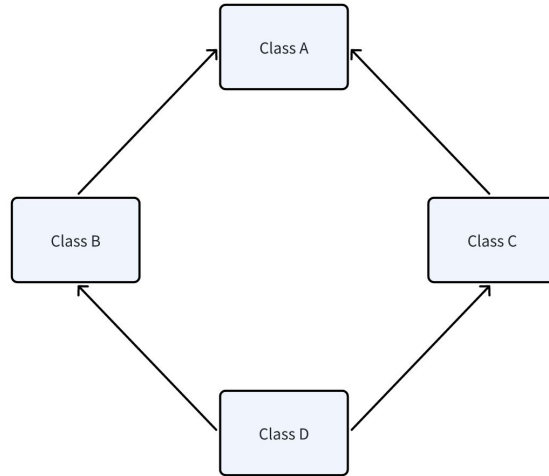


this vs super

	this	super
Refers to	The current class (subclass) object	The immediate parent class (superclass) object
Field reference	this.<field> refers to the current instance variable	super.<field> refers to the immediate superclass's instance variable
Constructor reference	this(...) refers to another constructor in the current class	super(...) refers to a constructor in the superclass
Method reference	this.<method>() refers to a method in the current class	super.<method>() refers to a method in the superclass
Other	Can be passed as an argument in a method call or be returned in a method	

Why Not Multiple Inheritance ?

- The Diamond Problem
 - An ambiguity that can arise as a consequence of allowing multiple inheritance



Interface

- A contract of behaviors that implementing classes must adhere to
- Syntax
 - `public interface Shape {}`
 - `public Triangle implements Shape {}`
- Java allows **multiple** inheritance in interface

Interface

- Like a class:
 - An interface can have methods and variables
 - Methods declared in interface are by default abstract
 - Only method signature, no body
- It's the blueprint of a class
 - Specifies what a class must do but not how to do it
- Specifies a set of methods that the class has to implement
- If class implements interface but doesn't provide method bodies for all functions specified in the interface, then class must be declared abstract

Aggregation / Composition

- Inheritance is used when a class is a specialized version of another (**IS-A**)
- Many relationships are described through **HAS-A** relationship
- If a class have an entity reference, it is known as **Aggregation** or **Composition**
- Aggregation represents **HAS-A** relationship
 - Employee object contains many information such as:
 - String id
 - String name
 - String email
 - **Address address**
 - String city
 - String zipcode
 - In such case, Employee has an entity reference address, so relationship is Employee **HAS-A** address

Why use aggregation

- Aggregation promotes code reusability when there is no IS-A relationship
- Without aggregation, code is repetitive and hard to maintain
 - If we don't have the Address object
 - Have to add all related information such as city and zipcode in employee
 - If we introduce another object called Company later which has the address too, we will have to add the same attributes to Company object again
 - If we make any changes (zipcode → postalCode), it may bring inconsistency.
- Aggregation provides flexibility and independence
 - Address object exists independently from Employee or Company

Aggregation

- How does Aggregation solve the diamond problem?
 - Instead of extending class B and C, introducing the B and C as the instance variables in class D
 - We can use `b.doSomething()` or `c.doSomething()` to eliminate the ambiguity

Inheritance vs. Aggregation

- Inheritance should be used only if the IS-A relationship is maintained throughout the lifetime of the objects involved; Otherwise, use aggregation
 - Flexibility of aggregation
 - For example, the method return type changed in super class:
 - With inheritance, we have no choice but to change our implementation
 - With aggregation, simply change the type of variable which stores the return value of super class
 - Unit testing (Mocking) - will introduce in the future

OOP Pillar: Polymorphism

- To perform a **single action in different ways** in Java
 - The word “poly” means many and “morphs” means forms
 - Hence, polymorphism means “many forms”
- Subclasses can define their own unique behaviors and share the same functionality of the superclass
- There are two types of polymorphism in Java:
 - Compile time polymorphism (Overloading)
 - Runtime polymorphism (Overriding)

Compile Time vs. Runtime

- Compile time: the instance where the code you entered is converted to executable
 - For example: from file.java to file.class
- Runtime: the instance where the executable is running
 - For example: when the for loop gets executed

Compile Time Polymorphism

- The “form” is determined at compile time
- Method **overloading**
 - A class has multiple methods having *same name* but *different signature* (in parameters)
 - Different **number** of parameters
 - Different data **type** of parameters
 - Method overloading increases the readability of the program

Runtime Polymorphism

- The “form” is determined at runtime
- Method **overriding**
 - A method in a **subclass** has the **same name, same parameters** and **return type** as the method in its **superclass**
 - Then the method in the subclass is said to **override** the method in the superclass
 - When an overridden method is called through the subclass object, it will always refer to the version of the method defined by the subclass
 - The superclass version of the method is overridden
- Static method hiding:
 - When a subclass defines a static method with the same signature as a static method in the superclass

Polymorphism

```
class Dog {  
    public void bark() {  
  
        System.out.println("woof");  
    }  
}
```

```
class Hound extends Dog {  
    // Overriding  
    // Same method name  
    // Same parameters  
    public void bark() {  
  
        System.out.println("woof");  
    }  
}
```

```
class Dog {  
    public void bark() {  
        System.out.println("woof");  
    }  
}
```

```
class Hound extends Dog {  
    // Overloading  
    // Same method name  
    // Different parameters  
    public void bark(int num) {  
        for (int i = 0; i < num; i++)  
        {  
            System.out.println("woof");  
        }  
    }  
}
```

Overloading vs. Overriding

Method Overloading	Method Overriding
Increases the readability of the code	Provides specific implementation of the method already provided by its superclass
Performed within a class	Occurs in two classes that have an IS-A relationship (inheritance)
Parameters must be different	Parameters must be the same
Compile time polymorphism	Runtime polymorphism
Parameters must be changed, return type can be either the same or different	Return type must be the same or covariant

OOP Pillar: Abstraction

- What:
 - Hiding internal details and showing functionality
 - For example, making a phone call, we don't know the internal processing
- How:
 - By creating either **Abstract Classes** or **Interfaces**
- Why:
 - Hide unnecessary things from user providing easiness
 - Hiding the internal implementation of software providing security

Abstract Class

- Abstract classes are classes with a generic concept, not related to a specific class
- Abstract classes define partial behavior and leave the rest for the subclasses to provide
- Contain zero or more abstract methods
- Abstract method contains no implementation (like method in interface)
- Abstract classes cannot be instantiated, but they can have reference variable
- If the subclasses doesn't override the abstract methods of the abstract class
 - It is **mandatory** for the subclasses to tag itself as **abstract**

Why Abstract Class

- To force same name and signature pattern in all the subclasses
- To have the flexibility to code these methods with their own specific requirements
- To prevent accidental initialization
- To define common attributes or methods

Abstract Class vs. Interface

Interface	Abstract Class
Multiple inheritance possible, a class can inherit any number of interfaces	Multiple inheritance not possible, a class can inherit only one class
implements keyword is used to inherit and interface	extends keyword is used to inherit a class
By default, all methods in an interface are public and abstract, no need to tag them as public and abstract	Methods can be public, protected, or package-private and can be abstract or concrete
Interfaces have no implementation at all (but from Java 8, interfaces can have default and static methods)	Abstract classes can have partial or full implementation of some methods
All methods of an interface need to be overridden unless they are default or static	Only abstract methods need to be overridden
All variables in an interface are implicitly public, static, and final	Variables can have any access modifier and do not have to be static or final
Interfaces do not have constructors	Abstract classes can have constructors

OOP Pillar: Encapsulation

- What:
 - Encapsulation means combining fields and methods into a single unit (class)
 - Encapsulation is hiding information → Use access modifier to do data hiding
- How:
 - Making the fields in a class private and providing access to the fields via public getter methods
- Why:
 - Data hiding: Encapsulation protects sensitive data from unauthorized access
 - Reusability: Encapsulated code can be used by different callers
 - Maintainability: Operations on encapsulated unit won't affect other parts
 - Flexibility: internal logic changes won't affect the caller of the method

Abstraction vs. Encapsulation

- Encapsulation bundles data and methods within one class, and control access to the data
- Abstraction hides the complex implementation details, only exposes necessary features
- Imagine you are using your phone
 - Abstraction: When you tap Call button → phone dials.
 - Encapsulation: phone OS, chip, battery inside the phone

Immutable class

- An object whose internal state cannot be changed after construction
- Eg: String, Wrapper classes (Integer, Long, Double, ..)
- Benefits
 - Thread safe, no synchronization needed (future lecture)
 - Safe for use as keys in HashMap / HashSet (future lecture)
 - Prevents accidental side effects
 - Easier reasoning, testing, and debugging
 - Natural support for functional programming style
- When to use:
 - Configuration objects
 - In multi-threaded environment
 - Keys

How to create an Immutable class

- Do not allow subclass to override by
 - Make class final $\Rightarrow$ Cannot be extended
 - Make the constructor private or use a factory method to create new instances
- All fields must be private and final
- Initialize all fields via constructor (deep copy if mutable objects)
- Do not provide setter methods, or methods that can change the property's values
- If fields are references to other objects:
 - Create a deep copy of the fields and operate on the copy.
 - Return the copy if necessary

A large red square with a white border, centered on a white background. The text "Thank You!" is written in white inside the red square.

Thank You!